# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 2 - Architecture Design Assignment

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 - Architecture Design Assignment |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills | | |
| **Student Name:** | **Mohamed Saif A** | **Reg. No.:** | **192521171** |
| **Date:** | **27/10/26** | **Duration:** | 60 minutes |

**Code of Conduct**
I Mohamed Saif A - 192521171 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
**Signature of the Student:** *Mohamed Saif A*

---

## Annexure A - Architecture Design Assignment

Based on your assigned problem statement, perform an Architecture Design Analysis and prepare an architecture design report by answering the following questions:

**Architecture Design of "ChargeGrid" - A Cloud-Based EV Charging Network Management and Smart Billing Platform**

**Problem Statement:** A city-wide network of electric-vehicle charging stations needs a scalable, always-available platform that connects thousands of chargers, lets drivers find, reserve, start and pay for charging, balances power against grid limits, detects faults in real time and bills accurately - even when chargers temporarily lose connectivity.

## 1. Analyze System Requirements

### 1.1 Study the problem statement and identify the system objectives

ChargeGrid must manage a large, geographically spread fleet of charging stations (EVSEs) and serve the drivers, operators and utilities that depend on them. It is a **real-time, IoT-heavy, payment-critical** platform: thousands of chargers hold permanent connections and stream meter readings every few seconds, drivers expect a charger to start within seconds of scanning a QR code, and every kilowatt-hour must be billed correctly. Chargers sit in basements and highways where network quality is poor, so the system must tolerate disconnections.

1. **Connect and manage chargers:** register stations, accept OCPP connections, monitor status, send remote commands (start, stop, reset, update firmware).
2. **Driver journey:** locate available chargers on a map, reserve a connector, authenticate (app, QR, RFID), start and stop sessions and view live charging status.
3. **Accurate metering and billing:** record energy delivered, apply dynamic tariffs and taxes, handle wallet / UPI / card payments, generate invoices and refunds.
4. **Smart load management:** keep station and grid load within limits by scheduling or throttling charging power, using utility demand-response signals.
5. **Reliability and fault handling:** detect charger faults, fraud and anomalies in real time; alert operators and support agents.
6. **Operator and fleet tools:** dashboards for uptime, utilisation, revenue and energy; tariff configuration; fleet-wise reports.
7. **Extensibility:** support new charger vendors, roaming with other networks (OCPI), vehicle-to-grid and plug-and-charge (ISO 15118) in future.

### 1.2 Functional requirements that influence the architecture

| ID | Functional Requirement | Architectural Influence |
|---|---|---|
| FR1 | Register users, authenticate by phone OTP / social login, manage roles (driver, operator, fleet manager, support, admin) | Identity & Access service, OIDC identity provider, RBAC, JWT at gateway |
| FR2 | Onboard chargers; maintain status; accept OCPP messages (BootNotification, Heartbeat, StatusNotification, MeterValues, Start/StopTransaction) | Dedicated OCPP WebSocket gateway and Device Communication service handling persistent connections |
| FR3 | Start / stop a charging session remotely or via QR / RFID; show live energy, power and cost | Charging Session service; low-latency command path; Redis for live state |
| FR4 | Search nearby chargers, reserve connectors, apply dynamic pricing | Reservation & Pricing service; cached station availability; maps integration |
| FR5 | Compute bills from meter values; wallet, UPI and card payments; invoices and refunds | Billing & Payments service; event-driven billing worker; payment adapter (tokenised, no card data stored) |
| FR6 | Grid-aware load balancing and demand-response participation | Smart Load Manager consuming telemetry and utility signals (OpenADR) |
| FR7 | Real-time fault, fraud and anomaly detection with alerts; push / SMS notifications | Stream processing workers on Kafka; Notification service |
| FR8 | Dashboards, utilisation and revenue reports, audit history of tariffs, commands and payments | Time-series and analytics stores; immutable Audit service |

### 1.3 Non-functional requirements that influence the architecture

| Category | Measurable Requirement | Architectural Influence |
|---|---|---|
| Performance | Remote start / stop acknowledged within 2 s (p95); live status refresh within 3 s; map search under 500 ms; payment confirmation under 5 s | Persistent WebSocket connections, Redis cache, async billing, geo-indexed station data |
| Scalability | 20,000 chargers and 100,000 drivers initially; sustained 700 messages/s with peaks of 2,000 messages/s (heartbeats + meter values every 30-60 s); 10x growth without redesign | Stateless services, horizontally scalable connection gateway, partitioned Kafka topics, time-series database |
| Availability | 99.95% platform availability; charging must continue when the cloud is unreachable | Multi-AZ deployment, replicated stores, local operation and offline transaction buffering at the charger |
| Reliability / Durability | No lost or double-billed transactions; meter data retained for dispute resolution; RPO <= 5 min, RTO <= 30 min | At-least-once events with idempotent billing, transaction replay after reconnection, backups |
| Security | mTLS for chargers (OCPP security profile 3); signed firmware; MFA for operators; PCI-DSS scope reduced through payment-gateway tokenisation; protection from rogue chargers and replay | Certificate management, WAF, API gateway, secrets vault, least-privilege roles |

| | | |
|---|---|---|
| Maintainability / Interoperability | Add a new charger vendor or tariff rule within one sprint; support OCPP 1.6J and 2.0.1, REST, OIDC and OCPI | Protocol adapters behind stable internal contracts, independent services, CI/CD |
| Auditability / Compliance | Every command, tariff change, session and payment logged immutably; data retention per regulation and privacy policy | Append-only audit store; retention policies; encryption of personal data |

### 1.4 Key quality attributes (ranked by influence on the architecture)

| Rank | Quality Attribute | Why it dominates in ChargeGrid |
|---|---|---|
| 1 | **Availability & Reliability** | A driver stranded at a dead charger or a lost transaction damages trust and revenue; the network has no maintenance window. |
| 2 | **Scalability & Performance** | Constant telemetry from thousands of devices plus evening-peak driver traffic require elastic, low-latency processing. |
| 3 | **Security** | Chargers are physical devices in public places connected to payments and the power grid; compromise has safety and financial impact. |
| 4 | **Maintainability & Interoperability** | Many charger vendors, protocol versions, tariffs and partner networks must be supported over a long hardware lifetime. |
| 5 | **Auditability** | Energy billing is a regulated, disputable transaction that must be provable. |

## 2. Select a Suitable Software Architecture

### 2.1 Selected architectural style

**Hybrid Architecture = Layered Client-Server + Domain Microservices + Event-Driven (stream) Processing with an Edge / IoT Gateway.** Mobile and web clients call domain microservices through an API gateway; chargers connect separately through an OCPP WebSocket gateway that terminates mTLS connections and hands messages to the Device Communication service. Every important fact (meter value, session started / stopped, fault detected, invoice generated) is published as an event to Apache Kafka, where independent workers handle telemetry ingestion, billing, anomaly detection and notifications. Each service owns its data store, and the system runs in containers (Kubernetes) across multiple availability zones. At the charger, an edge controller keeps working offline and replays buffered transactions on reconnection.

### 2.2 Evaluation of candidate architectural styles

| Style | Handles IoT streams / scale | Availability / fault isolation | Independent evolution | Complexity / cost | Verdict |
|---|---|---|---|---|---|
| Single layered monolith (MVC) | **Poor - connection handling and billing scale together** | **One failure takes all down** | **Low** | **Low** | Rejected: cannot isolate the connection load from billing |
| Client-Server only (2-tier) | **Poor** | **Poor** | **Low** | **Low** | Rejected: no support for device streaming or growth |
| Serverless (FaaS) only | **Good for events, poor for long-lived WebSockets** | **Good** | **Good** | **Cold starts, connection limits, vendor lock-in** | Partly used for light jobs, not for the OCPP gateway |
| Classic SOA with ESB | **Medium** | **ESB bottleneck** | **Medium** | **Heavy middleware** | Rejected: central bus becomes a bottleneck |

| | | | | | |
|---|---|---|---|---|---|
| Pure event-driven only | **Excellent** | **Good** | **Good** | **Hard to reason about request-response flows (login, start charging, payment)** | Used only where asynchrony fits |
| **Hybrid: Layered + Microservices + Event-Driven + Edge** | **Excellent** | **Excellent - isolation, multi-AZ, offline edge** | **Excellent** | **Moderate (reduced by managed Kubernetes / Kafka)** | **Selected** |

### 2.3 Why the selected architecture is the most suitable

- **Two very different workloads.** Chargers create a continuous, device-driven telemetry stream; drivers create bursty request-response traffic. Separate gateways and services let each scale on its own metric (connections vs. requests).
- **Event-driven billing is safe and auditable.** Meter values and session events are durable in Kafka and can be replayed; idempotent billing workers prevent double charging after retries.
- **Fault isolation.** If anomaly detection or notifications fail, charging and billing continue; if the cloud is unreachable, the edge controller finishes sessions locally.
- **Fits heterogeneous devices.** A Device Communication service with protocol adapters (OCPP 1.6J / 2.0.1) isolates vendor differences from business services.
- **Right store for each data type:** relational data (users, invoices), time-series data (meter values), in-memory state (live status) and immutable logs (audit).
- **Technology freedom and long life:** services can be upgraded independently over the 10-year life of charging hardware.

## 3. Draw the Software Architecture Diagram

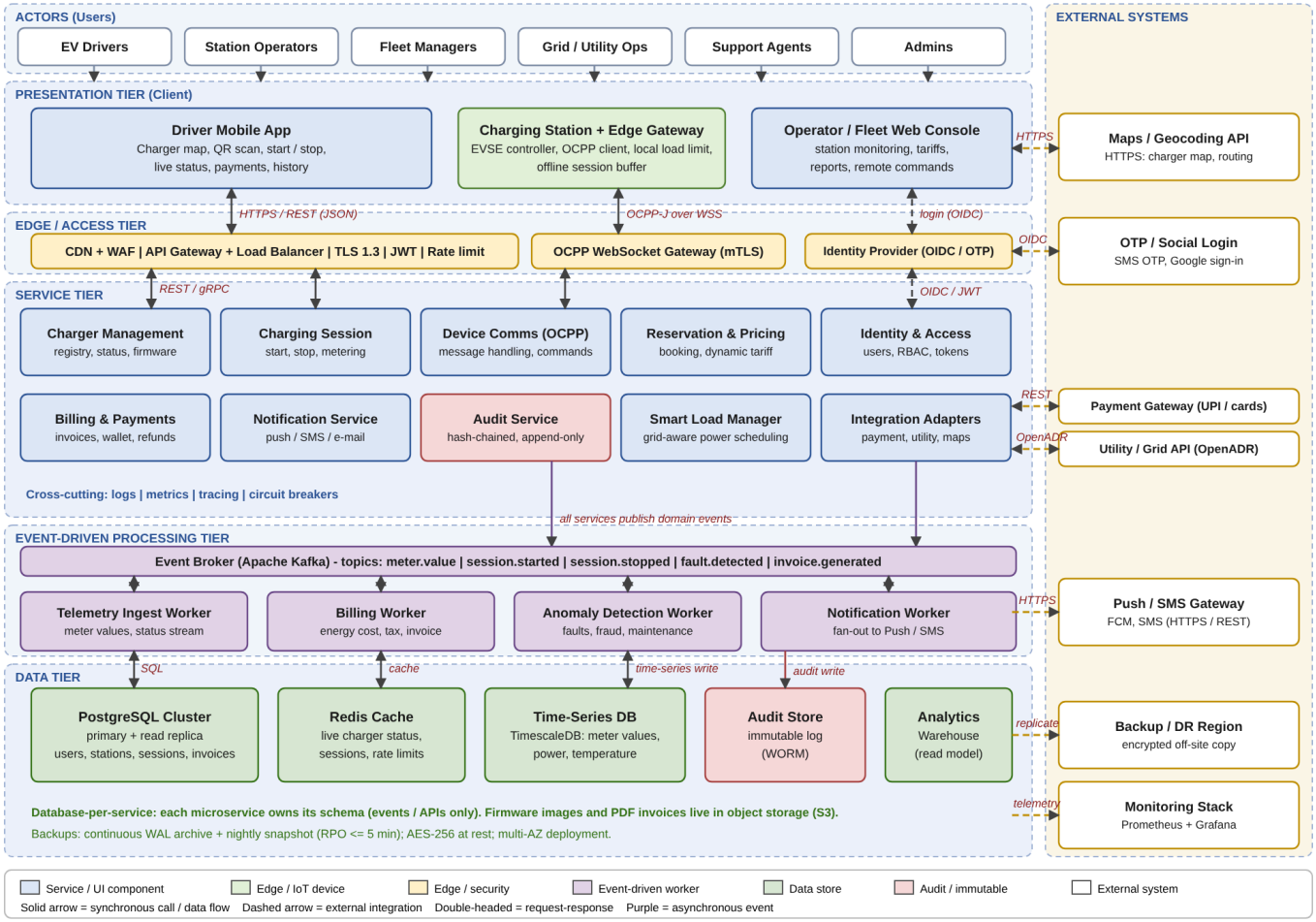



*Figure 1: Layered, microservice and event-driven architecture of the ChargeGrid platform*

### 3.1 Notation and reading guide

- **Tiers (dashed bands):** Actors, Presentation (apps, consoles and charging stations), Edge/Access, Service, Event-Driven Processing and Data tiers; external systems are shown in the right-hand column.
- **Boxes:** blue = business services / UI, green (top) = edge / IoT device, yellow = security and edge components, purple = event broker and asynchronous workers, green (bottom) = data stores, red = audit / immutable components, white = external systems.
- **Arrows:** solid double-headed = synchronous request-response (HTTPS / REST / gRPC / OCPP-J over WSS / SQL); purple = asynchronous domain events on Kafka; dashed = external integrations (OIDC, REST, OpenADR, HTTPS, replication, telemetry). Labels on the arrows state the protocol or data carried.

## 4. Explain Components and Their Interactions

### 4.1 Purpose and responsibilities of each component

| Component | Responsibility | Technology (proposed) |
|---|---|---|
| **Driver Mobile App** | Charger map and search, QR scan, reservation, start / stop, live charging status, wallet and payment, history | Flutter / React Native, push notifications |
| **Operator / Fleet Web Console** | Station monitoring, remote commands, tariff and fleet configuration, reports and dashboards | React SPA |
| **Charging Station + Edge Gateway** | Controls the connector, meters energy, speaks OCPP, enforces a local power limit and buffers transactions while offline | EVSE controller firmware, OCPP 1.6J / 2.0.1 client |
| **CDN + WAF + API Gateway + Load Balancer** | Terminates TLS, filters attacks, validates JWT, applies rate limits, routes REST traffic | NGINX / Kong, cloud WAF |

| | | |
|---|---|---|
| **OCPP WebSocket Gateway** | Terminates mTLS WebSocket connections from chargers, authenticates certificates, forwards messages and commands | Scalable WebSocket servers with sticky routing |
| **Identity Provider + Identity & Access Service** | Phone OTP / social login, token issue, roles and permissions, operator MFA | Keycloak (OIDC), JWT |
| **Charger Management Service** | Charger and connector registry, status, configuration, firmware campaigns, certificate lifecycle | Spring Boot, PostgreSQL |
| **Device Communication Service (OCPP)** | Maps OCPP messages to internal events and internal commands to OCPP calls; manages message queues per charger | Java / Go, Redis, Kafka |
| **Charging Session Service** | Authorises and tracks sessions, coordinates start / stop, stores session state and meter totals | Spring Boot, Redis, PostgreSQL |
| **Reservation & Pricing Service** | Connector reservations, dynamic and time-of-day tariffs, idle fees | Spring Boot, PostgreSQL |
| **Billing & Payments Service** | Wallet, invoice rules, refunds, payment orchestration through the gateway (tokenised) | Spring Boot, PostgreSQL |
| **Smart Load Manager** | Allocates available power across connectors and stations; reacts to grid signals | Python / Go service, OpenADR client |
| **Notification Service / Integration Adapters** | Sends push / SMS / e-mail; wraps payment, utility and maps APIs behind stable interfaces | REST adapters, FCM, SMS API |
| **Audit Service & Audit Store** | Receives commands, tariff changes, sessions and payments; writes an append-only, hash-chained record | WORM store / append-only table |
| **Event Broker + Workers** | Kafka decouples services; Telemetry Ingest, Billing, Anomaly Detection and Notification workers scale with queue lag | Apache Kafka, containerised consumers |
| **Data stores** | PostgreSQL (primary + replica), Redis (live state), TimescaleDB (meter values), analytics warehouse (read model), audit store (immutable), object storage (firmware, invoices) | PostgreSQL, Redis, TimescaleDB, S3 |

### 4.2 How components communicate and exchange data

| From - To | Mechanism | Data exchanged |
|---|---|---|
| Mobile app / console - API Gateway | HTTPS / REST (JSON), TLS 1.3 | Login, charger search, reserve, start / stop, payments with JWT |
| Charger - OCPP Gateway | OCPP-J over WSS with mutual TLS | Boot, heartbeat, status, meter values, transaction start / stop; remote commands in reverse |
| Gateway - Services | REST / gRPC with service discovery | Authenticated requests with user claims |
| Services - Identity Provider | OIDC / JWT validation | Tokens, roles, public keys |
| Services - Event Broker | Kafka publish / subscribe | Events: meter.value, session.started, session.stopped, fault.detected, invoice.generated |
| Workers - Data stores | SQL, cache protocol, time-series writes | Aggregated energy, invoices, fault records, live status |
| Adapters - Payment gateway / Utility / Maps | REST over HTTPS, OpenADR | Payment tokens and status; demand-response signals; geocoding and routing |
| Notification Worker - Push / SMS gateway | HTTPS | Charging complete, payment receipt, fault alerts |

| | | |
|---|---|---|
| All services - Audit Service | Async events | Who did what, when, from which device or IP; hash of previous record |

### 4.3 Overall workflow of the system

**Charger boot & registration** -> **Driver login & search** -> **Reserve / scan QR** -> **Authorise & start** -> **Live metering & load control** -> **Stop** -> **Billing & payment** -> **Invoice & notification**

1. **Onboarding:** The operator registers the charger; it boots, connects to the OCPP gateway with its certificate and is marked available.
2. **Login and search:** The driver signs in with OTP; the app calls the gateway, which validates the JWT; availability is read from the Redis-cached station status and shown on the map.
3. **Reserve / start:** The driver reserves a connector or scans the QR; the Charging Session service checks the wallet, applies the tariff and sends a RemoteStartTransaction through Device Communication; the charger acknowledges within 2 s.
4. **Charging:** The charger streams MeterValues; the Telemetry Ingest worker writes them to the time-series database and updates live status; the app shows kWh, power and cost in near real time.
5. **Load control:** The Smart Load Manager compares total demand with station and grid limits (OpenADR) and sends charging-profile commands to throttle or schedule power.
6. **Monitoring:** The Anomaly Detection worker consumes the stream; on a fault or suspicious pattern it publishes fault.detected and the operator is alerted.
7. **Stop and bill:** On stop (driver, full battery or fault) the charger sends StopTransaction; session.stopped triggers the Billing worker, which computes the invoice from final meter values (idempotently) and charges the payment method through the gateway adapter.
8. **Close:** Invoice PDF is stored, a notification is sent, analytics are updated and every step is written to the audit store. If the charger was offline, buffered transactions are replayed on reconnection and processed in the same way.

## 5. Discuss Quality Attributes

| Attribute | How the architecture addresses it (tactics) | Target / evidence |
|---|---|---|
| **Scalability** | Stateless services behind load balancers scale horizontally (Kubernetes HPA); OCPP gateway scales on open connections with consistent routing; Kafka partitions keyed by charger ID allow parallel consumers; time-series database partitions by time and charger; Redis absorbs hot reads; read replicas and a separate analytics store keep dashboards off the transaction database. | 20,000 chargers now; 200,000 by adding gateway nodes and partitions |
| **Security** | mTLS and per-charger certificates (OCPP security profile 3); signed firmware and secure boot; OIDC with OTP for drivers and MFA for operators; RBAC; WAF and rate limiting; card data never stored (payment-gateway tokenisation); encryption at rest (AES-256) and in transit (TLS 1.3); secrets vault; hash-chained audit log; anomaly detection for fraud and rogue devices. | OWASP / IEC 62443-aligned controls; no card data in scope |
| **Performance** | Persistent connections avoid reconnect cost; Redis for live status and session state; asynchronous billing and analytics off the critical path; geo-indexed station queries; batching of meter values into the time-series store; CDN for app assets. | Start / stop ack < 2 s; map search < 500 ms; status refresh < 3 s |
| **Reliability** | Edge controller continues sessions offline and buffers transactions; idempotent, at-least-once event processing; Kafka retention enables replay; circuit breakers, timeouts and retries between services; database replication and WAL archiving; reconciliation job compares meter totals with invoices daily. | No lost or double-billed transactions; RPO <= 5 min; RTO <= 30 min |
| **Maintainability** | Domain-aligned services with small codebases; protocol adapters isolate vendor differences; versioned APIs and event schemas (schema registry); independent CI/CD with canary releases; tariffs and load policies stored as configuration; centralised logging, metrics and tracing; infrastructure as code. | New vendor or tariff rule delivered without touching unrelated services |

| | | |
|---|---|---|
| **Availability** | Multi-AZ deployment with at least two replicas per service; health checks and automatic restart; managed PostgreSQL failover; replicated Kafka and Redis; DR region with backups; rolling and blue-green deployments with no downtime; local charging fallback at the station; 24x7 alerting and runbooks. | 99.95% platform availability; no single point of failure |

## 6. Justify Architectural Decisions

### 6.1 Rationale behind the architectural choices

| ID | Decision | Rationale | Alternative considered |
|---|---|---|---|
| D1 | Separate OCPP WebSocket gateway for chargers | Long-lived device connections scale differently from REST traffic and need mTLS and sticky routing | Serving chargers through the REST gateway (poor connection handling) |
| D2 | Domain microservices instead of a monolith | Independent scaling and failure isolation for sessions, billing and device communication | Modular monolith (simpler, but couples scaling and releases) |
| D3 | Kafka event backbone for telemetry, billing and alerts | Durable, replayable, high-throughput stream; decouples producers from consumers | Direct calls (cascading failures); simple queue (no replay) |
| D4 | Offline-capable edge controller with transaction replay | Chargers lose connectivity; charging and billing evidence must survive | Cloud-dependent charging (sessions fail offline) |
| D5 | Polyglot storage: PostgreSQL, Redis, TimescaleDB, WORM audit | Each data type gets the right consistency, speed and cost | One relational database for everything (slow time-series, contention) |
| D6 | Idempotent event-driven billing with daily reconciliation | Guarantees exactly-once billing effect on top of at-least-once delivery | Synchronous billing at session end (lost on failure) |
| D7 | Payment and utility integrations through adapters | Isolates the core from third-party changes; tokenisation keeps card data out of scope | Direct calls from each service |
| D8 | Kubernetes, multi-AZ deployment | Auto-scaling, self-healing, rolling updates, portable across clouds | Fixed VMs (manual scaling and recovery) |

### 6.2 Trade-offs considered

| Trade-off | Cost accepted | Mitigation |
|---|---|---|
| Microservices vs. simplicity | More operational complexity, network latency, harder debugging | About 8 coarse-grained services; managed Kubernetes / Kafka; distributed tracing; service templates |
| Eventual vs. strong consistency | Dashboards and analytics lag live data by seconds | Strong consistency only for session authorisation and payment; eventual consistency for derived data with clear status in the UI |
| Edge autonomy vs. central control | A charger may start a session offline without the latest cloud rules (local whitelist, power limit) | Cached authorisation lists with expiry, conservative local limits, reconciliation after reconnect |
| Cost vs. availability | Multi-AZ replicas and DR region raise infrastructure cost | Auto-scale non-critical services; right-size replicas; reserved capacity only for core path |
| Standards vs. vendor features | Relying on OCPP limits use of proprietary charger features | Vendor extensions through DataTransfer messages handled in adapters |

### 6.3 Support for future enhancements, integration and long-term maintainability

- **New capabilities as new consumers:** predictive maintenance, battery-health analytics or carbon dashboards can subscribe to existing Kafka topics without modifying current services (open-closed principle).
- **Roaming and partners:** OCPI adapters allow roaming with other charge-point operators; the adapter layer keeps partner protocols out of core logic.
- **Plug-and-charge and V2G:** ISO 15118 support can be added in the Device Communication service and Smart Load Manager without changing billing or the apps.

- **Multi-city / multi-tenant growth:** tenant IDs in tokens and data partitioning allow several operators to share one platform; additional regions can be added by replicating the stack.
- **Evolution path:** services are independently versioned, so a hot service can be split further or a rarely changed one merged; containers avoid cloud vendor lock-in.
- **Operational maintainability:** CI/CD, infrastructure as code, standard logs / metrics / traces, runbooks and architecture decision records keep the system understandable as teams change.

## Rubrics for Calculation

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **System Requirement Analysis (30%)** | Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture. | Analyzes most requirements with minor omissions or limited explanation. | Identifies only basic requirements with partial analysis. | Requirement analysis is incomplete, inaccurate, or lacks clarity. |
| **Selection of Software Architecture (25%)** | Selects the most appropriate architectural style with strong technical justification aligned to system requirements. | Selects a suitable architecture with reasonable justification. | Architecture is partially suitable with limited justification. | Inappropriate architecture selected or justification is missing. |
| **Architecture Diagram and Component Design (20%)** | Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented. | Diagram is mostly complete with minor omissions or notation issues. | Diagram includes only essential components and lacks sufficient detail. | Diagram is incomplete, unclear, or technically incorrect. |
| **Component Interaction and Quality Attribute Analysis (15%)** | Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability. | Explains most components and quality attributes with minor gaps. | Provides limited explanation of interactions and addresses only some quality attributes. | Component interactions and quality attributes are poorly explained or missing. |
| **Architectural Justification and Technical Presentation (10%)** | Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report. | Provides adequate justification with minor gaps; report is well organized. | Limited justification with minimal discussion of trade-offs or future scope; report needs improvement. | Justification is weak or absent; report lacks organization and technical clarity. |

| Criteria | Mark |
|---|---|
| System Requirement Analysis (30%) | /5 |
| Selection of Software Architecture (25%) | /5 |
| Architecture Diagram and Component Design (20%) | /5 |
| Component Interaction and Quality Attribute Analysis (15%) | /5 |
| Architectural Justification and Technical Presentation (10%) | /5 |
| **TOTAL** | **/25** |